Problem 7: Second-Best Score

TASK

A quiz app stores the scores of all players in an array. Prizes go to the top two different scores, so the app needs the second-highest distinct score. If every player has the same score, there is no second-highest score.

Implement a function `secondHighest(scores)` that returns the second-highest distinct score, or -1 if it does not exist.

CONSTRAINTS

- 1 <= length of scores <= 1000
- 0 <= scores[i] <= 100
- Do not sort the array

EXAMPLES

Sample 1

```
scores = [45, 78, 92, 78, 60]

Expected Output: 78
```

Sample 2

```
scores = [50, 50, 50]

Expected Output: -1
```

COMPLEXITY REQUIREMENTS

State the time and space complexity, and compare it with sorting the array first.

HINT

> *Keep two variables, highest and second, and update them as you visit each score once. Skip a score that equals the current highest.*

Problem 4: Exam Score Band Counter

TASK

A university stores the marks of all students in an exam as an array sorted in ascending order; marks can repeat. Faculty members frequently ask how many students scored within a band [low, high], inclusive of both ends.

Implement a function `countInBand(scores, low, high)` that returns the number of scores `s` such that `low <= s <= high`. The array is large and queried very often, so your solution must not scan the scores one by one.

CONSTRAINTS

- 1 <= length of scores <= 10^6
- scores is sorted in ascending order
- 0 <= low <= high <= 100

EXAMPLES

Sample 1

```
scores = [35, 42, 42, 50, 58, 58, 58, 63, 71, 88], low = 42, high = 58

Expected Output: 6
```

Sample 2

```
scores = [35, 42, 42, 50, 58, 58, 58, 63, 71, 88], low = 90, high = 100

Expected Output: 0
```

COMPLEXITY REQUIREMENTS

State the time and space complexity of a linear scan and of your final solution. Explain how duplicates at the edges of the band affect the way your search must be written.

HINT

> *A standard binary search stops at any matching element. Modify it to find the first position with a score >= low and the first position with a score > high; the answer is the difference.*

Problem 8: Duty Roster Rotation

TASK

A hostel rotates its cleaning duty roster every week by moving each name k places to the right; names that go past the end wrap around to the front.

Implement a function `rotateRoster(names, k)` that returns the roster after rotating it to the right by k places. k may be larger than the number of names.

CONSTRAINTS

- 1 <= length of names <= 100
- 0 <= k <= 10^9

EXAMPLES

Sample 1

```
names = [A, B, C, D, E], k = 2

Expected Output: [D, E, A, B, C]
```

Sample 2

```
names = [A, B, C, D, E], k = 7

Expected Output: [D, E, A, B, C]   // 7 places is the same as 2
```

COMPLEXITY REQUIREMENTS

State the time and space complexity of your solution. Explain why rotating one place at a time, k times, is slow when k is large.

HINT

> *Use k % n to remove full turns. Each name at index i moves to index (i + k) % n.*

Problem 10: Busiest Bus Row

TASK

A college bus records how many students sit in each seat as a 2D grid, where each row is one row of seats. The transport office wants to know which row carries the most students.

Implement a function `busiestRow(grid)` that returns the index of the row with the largest total and that total. If two rows tie, return the one with the smaller index.

CONSTRAINTS

- 1 <= rows, columns <= 10
- 0 <= grid[i][j] <= 3

EXAMPLE

```
grid = [[2, 0, 1],
        [3, 3, 1],
        [1, 1, 1]]

Expected Output: Row 1, Total 7
```

COMPLEXITY REQUIREMENTS

State the time complexity in terms of r (rows) and c (columns), and the additional space used.

HINT

> *Use an outer loop for the rows and an inner loop to add up the seats in each row; update the best row only when a total is strictly larger.*

Problem 6: Class Attendance Tracker

TASK

A teacher records attendance for one student as an array of 1s (present) and 0s (absent), one value per day. The teacher wants to know how many days the student was present and the longest run of days the student attended without a break.

Implement a function `attendanceSummary(days)` that returns the number of present days and the length of the longest unbroken run of present days.

CONSTRAINTS

- 1 <= length of days <= 100
- Each value is 0 or 1

EXAMPLES

Sample 1

```
days = [1, 1, 0, 1, 1, 1, 0, 1]

Expected Output: Present: 6, Longest streak: 3
```

Sample 2

```
days = [0, 0, 0]

Expected Output: Present: 0, Longest streak: 0
```

COMPLEXITY REQUIREMENTS

State the time and space complexity of your solution. Can both answers be found in a single pass?

HINT

> *Keep a running count that grows on every 1 and resets to 0 on every 0; remember the largest value it reaches.*